Module 8: Advanced TypeScript Features & Best Practices

1. Introduction to Advanced TypeScript Features

TypeScript provides advanced features for better type safety, code maintainability, and performance.

2. Utility Types in TypeScript

Utility types help manipulate types easily.

Partial<Type> (Makes all properties optional)

```
typescript
```

interface User {

name: string;

age: number;

}

const user: Partial<User> = { name: "Alice" };

Readonly<Type> (Makes all properties read-only)

```
typescript
```

const user: Readonly<User> = { name: "Alice", age: 25 };

// user.age = 30; // Error: Cannot assign to 'age' because it is a read-only property.

Pick<Type, Keys> (Selects specific properties)

```
typescript
```

type UserPreview = Pick<User, "name">;

const userPreview: UserPreview = { name: "Alice" };

Omit<Type, Keys> (Removes specific properties)

```
typescript
```

type UserWithoutAge = Omit<User, "age">;

const userWithoutAge: UserWithoutAge = { name: "Alice" };

3. Generic Functions and Classes

Generics provide flexibility while maintaining type safety.

Generic Function Example

```
typescript
```

```
function identity<T>(arg: T): T {
return arg;
}

console.log(identity<string>("Hello"));
console.log(identity<number>(42));
```

Generic Class Example

```
typescript
```

```
class Box<T> {
content: T;
constructor(content: T) {
this.content = content;
}
}

const numberBox = new Box<number>(100);
const stringBox = new Box<string>("TypeScript");
```

4. Type Guards and Type Assertions

Type guards help handle different types at runtime.

Using `typeof` for Type Guards

```
typescript
```

```
function printId(id: string | number) {
if (typeof id === "string") {
```

```
console.log("ID:", id.toUpperCase());
} else {
console.log("ID:", id.toFixed(2));
}
}
```

Type Assertions (Overriding TypeScript's Inference)

```
typescript
let someValue: any = "Hello, TypeScript";
let strLength: number = (someValue as string).length;
```

5. Advanced Mapped Types

Mapped types allow transforming object types.

```
typescript
type Optional<T> = { [K in keyof T]?: T[K] };
type ReadonlyUser = { readonly [K in keyof User]: User[K] };
```

6. TypeScript Decorators (Used in Angular & Class-based TS)

Decorators add metadata to classes, methods, or properties.

Class Decorator Example

```
typescript
function Logger(constructor: Function) {
console.log("Logging class creation...");
}

@Logger
class Person {
constructor(public name: string) {}
}
```

Method Decorator Example

```
typescript
function Log(target: any, methodName: string, descriptor: PropertyDescriptor) {
console.log("Logging method:", methodName);
}

class Car {
@Log
drive() {
console.log("Driving...");
}
}
```

7. Best Practices in TypeScript

- Use strict mode (`"strict": true` in `tsconfig.json`).
- Prefer `interface` over `type` where possible.
- Avoid `any` and use proper typing.
- Leverage utility types for cleaner code.
- Use `readonly` for immutable objects.

Practice Questions:

1. Create a function using TypeScript Generics that returns the last element of an array.
2. Define a mapped type that makes all properties in an object optional.
3. Implement a type guard function that checks whether a value is a string.
4. Use a class decorator to log the creation of an instance.
5. Use the `Omit<>` utility type to exclude a property from an interface.

Practice Answers:

1. Generic Function:

```
typescript
function lastElement<T>(arr: T[]): T {
return arr[arr.length - 1];
}

console.log(lastElement([1, 2, 3])); // 3
```

```
console.log(lastElement(["a", "b", "c"])); // "c"
```

2. Mapped Type for Optional Properties:

```
typescript
type Optional<T> = { [K in keyof T]?: T[K] };
```

3. Type Guard Function:

```
typescript
function isString(value: any): value is string {
return typeof value === "string";
}

console.log(isString("Hello")); // true
console.log(isString(42)); // false
```

4. Class Decorator:

```
typescript
function Logger(constructor: Function) {
console.log("Logging class creation...");
}

@Logger
class Person {
constructor(public name: string) {}
}
```

5. Using `Omit<>`:

```
typescript
interface User {
id: number;
name: string;
email: string;
}

type UserWithoutEmail = Omit<User, "email">;
const user: UserWithoutEmail = { id: 1, name: "Alice" };
```